LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : Reva Lina
NIM : 2428240105
Kelas : SI5B

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	Reva Lina
NPM	2428240105
Kelas	SI5B
Pertemuan Ke-	5
Topik / Judul Praktikum	RESTful API dengan Express.js: Routing, Middleware, dan CORS
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat, M.Kom.

I. Tujuan Praktikum

1. Memahami konsep dan prinsip RESTful API serta penggunaan metode HTTP (GET, POST, PUT) untuk operasi data.
2. Mampu membangun server web menggunakan Express.js dengan beberapa route, termasuk route dinamis (:id) dan query string.
3. Mampu menerapkan middleware (logger, express.json, dan cors) serta menyimpan konfigurasi dalam file .env menggunakan dotenv.
4. Mampu menguji API dan memahami kebijakan CORS melalui halaman frontend sederhana.

II. Dasar Teori

RESTful API. REST (Representational State Transfer) adalah gaya arsitektur untuk membangun layanan web di mana setiap data diperlakukan sebagai resource yang diakses melalui URL. Operasi terhadap resource dinyatakan dengan metode HTTP: GET untuk membaca, POST untuk menambah, PUT untuk memperbarui, dan DELETE untuk menghapus. Server membalas dengan data (umumnya JSON) beserta kode status HTTP, misalnya 200 (berhasil), 201 (data dibuat), 400 (permintaan tidak valid), dan 404 (data tidak ditemukan).

Node.js dan Express.js. Node.js adalah lingkungan runtime JavaScript di sisi server. Express.js adalah framework web minimalis di atas Node.js yang menyediakan routing dan middleware sehingga pembuatan API menjadi lebih ringkas.

Routing. Routing menentukan bagaimana aplikasi merespons permintaan ke suatu URL dan metode HTTP tertentu. Data dari klien dapat dibaca melalui parameter route (req.params, contoh /mahasiswa/:id), query string (req.query, contoh ?jurusan=...), dan isi permintaan (req.body).

Middleware. Middleware adalah fungsi yang berjalan di antara permintaan dan respons dan memiliki akses ke req, res, serta next(). Pada praktikum ini digunakan middleware buatan sendiri (logger untuk mencatat setiap request), express.json() agar body JSON dapat dibaca, dan cors untuk mengatur asal (origin) yang boleh mengakses API.

CORS dan variabel lingkungan. CORS (Cross-Origin Resource Sharing) adalah mekanisme keamanan peramban yang membatasi permintaan lintas asal. Server harus mengizinkan origin tertentu agar frontend di alamat lain dapat memanggil API. Konfigurasi seperti PORT dan CORS_ORIGIN disimpan di file .env dan dibaca dengan dotenv agar tidak ditulis langsung di kode; file .env tidak diunggah ke repository (dimasukkan ke .gitignore), sedangkan contohnya disediakan pada .env.example.

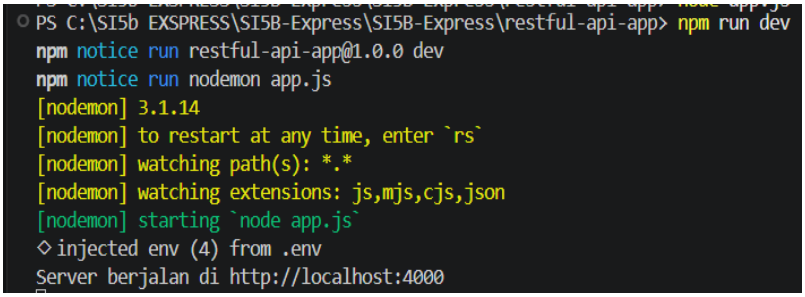
III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript sisi server (versi sesuai hasil perintah node -v)
2	Visual Studio Code	Editor kode
3	Express.js	Versi 5.2.1 (framework web)
4	cors, dotenv, nodemon	cors 2.8.6, dotenv 18.0.4, nodemon 3.1.14 (paket npm)
5	Peramban web	Menguji endpoint dan halaman Frontend-uji/index.html
6	Git dan GitHub	Version control dan penyimpanan repository

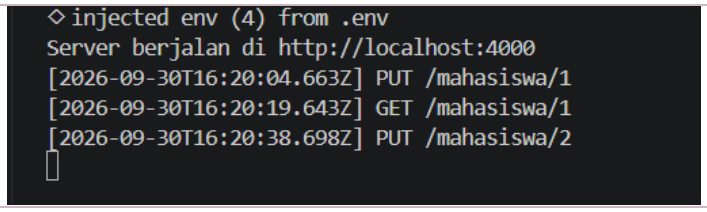
IV. Langkah Kerja

1. Membuat folder proyek restful-api-app, lalu menjalankan npm init untuk membuat package.json.
2. Memasang dependensi dengan perintah npm install express cors dotenv, serta npm install --save-dev nodemon. Pada package.json ditambahkan script "start": "node app.js" dan "dev": "nodemon app.js".
3. Membuat file .env (berisi PORT=4000 dan CORS_ORIGIN=http://localhost:5500) dan .env.example sebagai contoh, lalu menulis .gitignore yang mengabaikan node_modules/ dan .env.
4. Membuat app.js: memuat dotenv, membuat aplikasi Express, menulis middleware logger yang mencatat waktu, metode, dan URL setiap request, kemudian mendaftarkan logger, cors (origin dari CORS_ORIGIN, metode GET/POST/PUT/DELETE), dan express.json().
5. Membuat route dasar: GET / (mengirim teks "Server Express.js berjalan!") dan GET /profil, kemudian menjalankan server dengan app.listen pada PORT dari .env.
6. Membuat data sementara berupa array mahasiswa di memori dan endpoint CRUD: GET /mahasiswa (dengan filter ?jurusan=), GET /mahasiswa/:id, POST /mahasiswa (validasi nama dan jurusan, status 201), dan PUT /mahasiswa/:id (status 404 jika id tidak ada).
7. Menjalankan server dengan npm run dev, lalu menguji setiap endpoint melalui peramban atau aplikasi pengujian API dan memeriksa log yang muncul di terminal.
8. Membuat Frontend-uji/index.html yang memanggil http://localhost:4000/mahasiswa dengan fetch() untuk menguji CORS, lalu membandingkan hasilnya ketika origin diizinkan dan tidak diizinkan. Halaman dibuka melalui Live Server pada http://localhost:5500 sehingga sesuai dengan CORS_ORIGIN.
9. Mengunggah proyek ke repository GitHub.

V. Hasil dan Pembahasan



Gambar 1. Server Express berhasil berjalan di port 4000 menggunakan nodemon (npm run dev) dengan konfigurasi dari file .env.



Gambar 2. Log middleware logger di terminal yang mencatat setiap request (waktu, metode, dan URL), yaitu PUT dan GET pada endpoint /mahasiswa/:id.

Pembahasan:

Gambar 1 Pada terminal terlihat bahwa aplikasi dijalankan menggunakan nodemon, sehingga server akan otomatis melakukan restart ketika terdapat perubahan pada kode. Kemudian muncul injected env (4) from .env, yang menunjukkan bahwa terdapat 4 variabel konfigurasi yang berhasil dibaca dari file .env. Terakhir, muncul Server berjalan di http://localhost:4000, yang berarti server Express.js berhasil aktif dan dapat diakses melalui port 4000.

Pada gambar 2 terlihat server Express.js berjalan pada localhost:4000. Kemudian terdapat beberapa request yang masuk, yaitu PUT /mahasiswa/1, GET /mahasiswa/1, dan PUT /mahasiswa/2. Request PUT digunakan untuk memperbarui data mahasiswa, sedangkan request GET digunakan untuk mengambil data mahasiswa. Hal ini menunjukkan bahwa API berhasil menerima request dari client.

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Permintaan dari Frontend-uji diblokir peramban (error CORS).	Menyesuaikan nilai CORS_ORIGIN pada file .env dengan alamat asal halaman frontend, lalu menjalankan ulang server.

VII. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa:

1. RESTful API sederhana berhasil dibuat dengan Express.js menggunakan metode GET, POST, dan PUT untuk mengelola data mahasiswa, lengkap dengan kode status HTTP yang sesuai (200, 201, 400, 404). Hal ini memenuhi tujuan pertama dan kedua.
2. Route dinamis (:id) dan query string (?jurusan=) berhasil diterapkan untuk mengambil dan menyaring data (tujuan kedua).
3. Middleware logger, express.json, dan cors berhasil diterapkan, dan konfigurasi PORT serta CORS_ORIGIN dipisahkan ke file .env dengan dotenv sehingga kode lebih aman dan fleksibel (tujuan ketiga).
4. Pengujian dengan halaman frontend menunjukkan bahwa CORS menentukan apakah aplikasi di origin lain dapat mengakses API (tujuan keempat).

Daftar Pustaka

Express. (2026). Express - Node.js web application framework. https://expressjs.com/

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine].

Mozilla Developer Network. (2026). Cross-Origin Resource Sharing (CORS). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS

OpenJS Foundation. (2026). Node.js documentation. https://nodejs.org/docs/latest/api/

Lampiran

Tautan Repository GitHub:
https://github.com/revalina2428240105/si5b_latihan_restful_RevaLina
Tautan Deployment (jika ada):
https://vercel.com/revalina/si5b-latihan-restful-reva-lina